

1.6 — Design Task Decomposition Strategies for Complex Workflows

Concept

Complex workflows require breaking work into manageable subtasks. Two fundamentally different strategies exist, and choosing between them determines how much control you retain versus how much reasoning you delegate to the model.

Prompt Chaining (Static Sequential Decomposition)

Prompt chaining hardcodes the subtask sequence in your application logic. Each step's output feeds the next step's input. The developer decides upfront what the steps are, in what order they run, and what each step receives.

When to use: Tasks with a known, stable structure — e.g., extract → transform → validate → write. The workflow shape does not depend on the content of the data being processed.

Tradeoffs:

- Predictable and auditable: every run follows the same path
- Easy to unit-test each step in isolation
- Brittle when task structure varies by input — a pipeline built for three steps breaks if a fourth is sometimes needed
- Context passed between steps must be explicitly designed; information can be lost at handoff boundaries

Dynamic Decomposition (Model-Directed Subtasking)

Dynamic decomposition gives the model a high-level objective and lets it decide at runtime which subtasks to create, in what order, and when the goal is achieved. The orchestrator loop executes whatever the model requests via tools rather than following a fixed plan.

When to use: Tasks where the number and nature of subtasks cannot be known in advance — e.g., "refactor this codebase to remove all deprecated API calls" where the set of files, the number of call sites, and the required changes all depend on the codebase content.

Tradeoffs:

- Flexible and adaptive to arbitrary inputs
- Higher token cost — the model reasons about structure on every invocation
- Harder to test and audit: execution paths vary per run
- Requires robust tool design and loop termination logic to prevent runaway execution

Per-File Analysis Pattern (Independent Parallel Processing)

When subtasks are independent — each piece of work does not require output from other pieces — they can be executed in parallel. The per-file analysis pattern treats each file (or chunk) as a self-contained unit: read it, analyze it, produce a result, discard its context.

When to use: Linting, summarization, scoring, or classification where each item stands alone. The results are aggregated after all parallel units complete.

Key constraint: No subtask may read the output of a sibling subtask. Independence is the invariant that makes parallelism safe.

Cross-File Integration (Shared-Context Processing)

Some tasks require understanding relationships across files — a type defined in `types.ts` affects how `handler.ts` should be analyzed, for example. These tasks require shared context: either pass all relevant files in one large prompt, or maintain a running synthesis layer that accumulates findings before drawing conclusions.

When to use: Dependency analysis, architectural reviews, refactoring that must remain internally consistent, or any task where a decision in one file constrains another.

Tradeoffs:

- Context window pressure grows with the number of files
- Sequential rather than parallel — earlier results must be available before later analysis begins
- Requires careful context summarization to avoid prompt bloat

Granularity Tradeoffs

Finer granularity (smaller subtasks) improves parallelism and reduces per-step context requirements but increases coordination overhead and the number of API calls. Coarser granularity reduces overhead but risks hitting context limits and makes individual steps harder to test or retry.

A practical heuristic: decompose to the granularity where each subtask can be clearly specified and independently verified. If a subtask requires another subtask's output to be validated, they should be merged or sequenced explicitly.

Code Example

```
// Prompt chaining – static steps, developer controls flow
async function analyzeFileChain(filePath: string): Promise<AnalysisResult> {
  const content = await readFile(filePath);
  const summary = await summarize(content);           // Step 1
  const issues = await detectIssues(summary);         // Step 2 – receives Step 1 output
  const report = await formatReport(issues);          // Step 3 – receives Step 2 output
  return report;
}

// Per-file parallel analysis – independent units, no shared state
```

```

async function analyzeFilesInParallel(files: string[]): Promise<AnalysisResult[]> {
  return Promise.all(files.map(file => analyzeFileChain(file)));
  // Each call is independent – safe to parallelize
}

// Cross-file integration – shared context accumulated across files
async function analyzeWithSharedContext(files: string[]): Promise<IntegrationResult> {
  const sharedContext: string[] = [];

  for (const file of files) {
    const content = await readFile(file);
    // Each step receives the accumulated context from prior files
    const finding = await client.messages.create({
      messages: [{ role: 'user', content: `Context so far:\n${sharedContext.join('\n')}\n\nNow
    }]);
    sharedContext.push(extractText(finding));    // Grow shared context
  }

  return synthesize(sharedContext);
}

```

Common Mistakes

1. **Using dynamic decomposition for fixed workflows** — If the task structure is always the same, a hardcoded chain is cheaper, faster, and more testable. Reserve dynamic decomposition for genuinely variable workflows.
2. **Parallelizing dependent subtasks** — Running cross-file analysis in parallel produces incorrect results when subtask B requires subtask A's output. Verify independence before using `Promise.all`.
3. **Ignoring context accumulation cost** — In cross-file integration, naively appending every file's full content to the shared context blows the context window. Summarize aggressively between steps.
4. **Decomposing too finely** — Each additional subtask adds latency, token overhead, and a potential failure point. If two subtasks always run back-to-back with no branching between them, they belong in a single step.

References

- [Anthropic: Build with Claude — Orchestration Patterns](#)
- [Anthropic: Prompt Chaining](#)